

Collective Communications and Error Handling in MPI

Lab 4

Adi Ramanarayan

230905380 48

A2

Lab Exercises:

Q1. Sum of factorials using MPI collective communication with error handling.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

void check_mpi_error(int err, const char *msg) {
    if (err != MPI_SUCCESS) {
        char err_string[MPI_MAX_ERROR_STRING];
        int err_len;
        MPI_Error_string(err, err_string, &err_len);
        fprintf(stderr, "[MPI ERROR] %s: %s\n", msg, err_string);
        MPI_Abort(MPI_COMM_WORLD, err);
    }
}

long long factorial(int n) {
    long long fact = 1;
    for (int i = 2; i <= n; i++)
        fact *= i;
}
```

```

        return fact;
    }

int main(int argc, char** argv) {
    int err;

    err = MPI_Init(&argc, &argv);
    check_mpi_error(err, "MPI_Init failed");

    err = MPI_Comm_set_errhandler(MPI_COMM_WORLD, MPI_ERRORS_RETURN);
    check_mpi_error(err, "Setting error handler failed");

    int rank, size;
    err = MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    check_mpi_error(err, "MPI_Comm_rank failed");

    err = MPI_Comm_size(MPI_COMM_WORLD, &size);
    check_mpi_error(err, "MPI_Comm_size failed");

    int n = 0;

    if (rank == 0) {
        if (argc < 2) {
            fprintf(stderr, "Usage: mpirun -np <p> %s <n>\n",
argv[0]);
            MPI_Abort(MPI_COMM_WORLD, 1);
        }
        n = atoi(argv[1]);
        if (n < 1) {
            fprintf(stderr, "n must be >= 1\n");
            MPI_Abort(MPI_COMM_WORLD, 1);
        }
    }

    err = MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);
    check_mpi_error(err, "MPI_Bcast failed");

    long long local_sum = (rank < n) ? factorial(rank + 1) : 0;
    long long scan_sum = 0;

    err = MPI_Scan(&local_sum, &scan_sum, 1,
        MPI_LONG_LONG_INT, MPI_SUM, MPI_COMM_WORLD);
    check_mpi_error(err, "MPI_Scan failed");

    if (rank < n) {

```

```

        printf("Rank %d: %d! = %lld\n", rank, rank + 1, local_sum);
    }

    if (rank == n - 1) {
        printf("Sum of factorials (1! to %d!) = %lld\n", n,
scan_sum);
    }

    err = MPI_Finalize();
    check_mpi_error(err, "MPI_Finalize failed");

    return 0;
}

```

Output:

```

● STUDENT@MIT-ICT-LAB5-13:~/Desktop/230905380/Week4$ mpirun -np 5 ./q1 4
Rank 0: 1! = 1
Rank 1: 2! = 2
Rank 2: 3! = 6
Rank 3: 4! = 24
Sum of factorials (1! to 4!) = 33
○ STUDENT@MIT-ICT-LAB5-13:~/Desktop/230905380/Week4$ █

```

Q2. Matrix (3x3) read and count number of occurrences of entered value. Handle errors.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

void check_mpi_error(int err, const char *msg) {
    if (err != MPI_SUCCESS) {
        char err_string[MPI_MAX_ERROR_STRING];
        int err_len;
        MPI_Error_string(err, err_string, &err_len);
        fprintf(stderr, "[MPI ERROR] %s: %s\n", msg, err_string);
        MPI_Abort(MPI_COMM_WORLD, err);
    }
}

int main(int argc, char** argv) {
    int err;

    err = MPI_Init(&argc, &argv);
    if (err != MPI_SUCCESS) {
        fprintf(stderr, "MPI_Init failed\n");
        return EXIT_FAILURE;
    }

    err = MPI_Comm_set_errhandler(MPI_COMM_WORLD, MPI_ERRORS_RETURN);
    check_mpi_error(err, "Setting error handler failed");

    int rank, size;
    err = MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    check_mpi_error(err, "MPI_Comm_rank failed");

    err = MPI_Comm_size(MPI_COMM_WORLD, &size);
    check_mpi_error(err, "MPI_Comm_size failed");

    if (size != 3) {
        if (rank == 0) {
            fprintf(stderr, "Error: This program requires exactly 3
processes. Current size: %d\n", size);
        }
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
}
```

```

int matrix[3][3];
int row[3];
int element_to_search;
int local_count = 0;

if (rank == 0) {
    printf("Enter elements of 3x3 matrix:\n");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (scanf("%d", &matrix[i][j]) != 1) {
                fprintf(stderr, "Error: Invalid input for matrix
element.\n");
                MPI_Abort(MPI_COMM_WORLD, 1);
            }
        }
    }
    printf("Enter element to search: ");
    if (scanf("%d", &element_to_search) != 1) {
        fprintf(stderr, "Error: Invalid input for search
element.\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
}

err = MPI_Bcast(&element_to_search, 1, MPI_INT, 0,
MPI_COMM_WORLD);
check_mpi_error(err, "MPI_Bcast failed");

err = MPI_Scatter(matrix, 3, MPI_INT, row, 3, MPI_INT, 0,
MPI_COMM_WORLD);
check_mpi_error(err, "MPI_Scatter failed");

for (int j = 0; j < 3; j++) {
    if (row[j] == element_to_search) {
        local_count++;
    }
}

int total_count;
err = MPI_Reduce(&local_count, &total_count, 1, MPI_INT, MPI_SUM,
0, MPI_COMM_WORLD);
check_mpi_error(err, "MPI_Reduce failed");

if (rank == 0) {

```

```
        printf("Total occurrences of %d in the matrix: %d\n",
element_to_search, total_count);
    }

    err = MPI_Finalize();
    check_mpi_error(err, "MPI_Finalize failed");

    return 0;
}
```

Output:

```
● STUDENT@MIT-ICT-LAB5-13:~/Desktop/230905380/Week4$ mpirun -np 3 ./q2
Enter elements of 3x3 matrix:
4
2
1
1
7
4
2
0
6
Enter element to search: 4
Total occurrences of 4 in the matrix: 2
○ STUDENT@MIT-ICT-LAB5-13:~/Desktop/230905380/Week4$
```

**Q3. Matrix (4x4) read and replace column values with sum of current and all previous values.
Handle errors and display both matrices.**

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define N 4

void check_mpi_error(int err, const char *msg) {
    if (err != MPI_SUCCESS) {
        char err_string[MPI_MAX_ERROR_STRING];
        int err_len;
        MPI_Error_string(err, err_string, &err_len);
        fprintf(stderr, "[MPI ERROR] %s: %s\n", msg, err_string);
        MPI_Abort(MPI_COMM_WORLD, err);
    }
}

void read_matrix(int mat[N][N]) {
    printf("Enter elements of 4x4 matrix:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (scanf("%d", &mat[i][j]) != 1) {
                fprintf(stderr, "Error: Invalid input.\n");
                MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
            }
        }
    }
}

void display_matrix(int mat[N][N]) {
    printf("Output Matrix:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", mat[i][j]);
        }
        printf("\n");
    }
}

int main(int argc, char** argv) {
```

```

int err;
int rank, size;

int input_matrix[N][N];
int final_matrix[N][N];

int local_row[N];
int scan_result_row[N];

err = MPI_Init(&argc, &argv);
if (err != MPI_SUCCESS) {
    fprintf(stderr, "MPI_Init failed\n");
    return EXIT_FAILURE;
}

MPI_Comm_set_errhandler(MPI_COMM_WORLD, MPI_ERRORS_RETURN);

err = MPI_Comm_rank(MPI_COMM_WORLD, &rank);
check_mpi_error(err, "MPI_Comm_rank failed");

err = MPI_Comm_size(MPI_COMM_WORLD, &size);
check_mpi_error(err, "MPI_Comm_size failed");

if (size != 4) {
    if (rank == 0) {
        fprintf(stderr, "Error: This program requires exactly 4
processes (one per row).\n");
    }
    MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
}

if (rank == 0) {
    read_matrix(input_matrix);
    printf("Input Matrix:\n");
    for(int i=0; i<N; i++) {
        for(int j=0; j<N; j++) printf("%d ", input_matrix[i][j]);
        printf("\n");
    }
    printf("\nProcessing...\n");
}

err = MPI_Scatter(input_matrix, N, MPI_INT,
                  local_row, N, MPI_INT,
                  0, MPI_COMM_WORLD);
check_mpi_error(err, "MPI_Scatter failed");

```



```

    err = MPI_Scan(local_row, scan_result_row, N, MPI_INT, MPI_SUM,
MPI_COMM_WORLD);
    check_mpi_error(err, "MPI_Scan failed");

    err = MPI_Gather(scan_result_row, N, MPI_INT,
                    final_matrix, N, MPI_INT,
                    0, MPI_COMM_WORLD);
    check_mpi_error(err, "MPI_Gather failed");

    if (rank == 0) {
        display_matrix(final_matrix);
    }

    err = MPI_Finalize();
    check_mpi_error(err, "MPI_Finalize failed");

    return 0;
}

```

Output:

```

STUDENT@MIT-ICT-LAB5-13:~/Desktop/230905380/Week4$ mpirun -np 4 ./q3
Input Matrix:
1 2 3 4
1 2 3 1
1 1 1 1
2 1 2 1

Processing...
Output Matrix:
1 2 3 4
2 4 6 5
3 5 7 6
5 6 9 7
STUDENT@MIT-ICT-LAB5-13:~/Desktop/230905380/Week4$ █

```

Q4. Program to read a word (string) of length N. Use N processes to repeat each letter according to its position.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <mpi.h>

void check_mpi_error(int err, const char *msg) {
    if (err != MPI_SUCCESS) {
        char err_string[MPI_MAX_ERROR_STRING];
        int err_len;
        MPI_Error_string(err, err_string, &err_len);
        fprintf(stderr, "[MPI ERROR] %s: %s\n", msg, err_string);
        MPI_Abort(MPI_COMM_WORLD, err);
    }
}

void read_input_string(char *buffer, int max_len) {
    printf("Enter a word of length %d: ", max_len);
    if (scanf("%s", buffer) != 1) {
        fprintf(stderr, "Error reading input.\n");
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }
}

int main(int argc, char** argv) {
    int err;
    int rank, size;
    char *input_word = NULL;
    char *final_result = NULL;

    int *recvcounts = NULL;
    int *displs = NULL;

    err = MPI_Init(&argc, &argv);
    if (err != MPI_SUCCESS) {
        fprintf(stderr, "MPI_Init failed\n");
        return EXIT_FAILURE;
    }

    err = MPI_Comm_set_errhandler(MPI_COMM_WORLD, MPI_ERRORS_RETURN);
```

```

check_mpi_error(err, "Setting error handler failed");

err = MPI_Comm_rank(MPI_COMM_WORLD, &rank);
check_mpi_error(err, "MPI_Comm_rank failed");

err = MPI_Comm_size(MPI_COMM_WORLD, &size);
check_mpi_error(err, "MPI_Comm_size failed");

if (rank == 0) {
    input_word = (char *)malloc((size + 1) * sizeof(char));
    if (!input_word) {
        fprintf(stderr, "Malloc failed for input_word\n");
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }

    read_input_string(input_word, size);

    int len = strlen(input_word);
    if (len != size) {
        fprintf(stderr, "Error: Word length (%d) must match
number of processes (%d).\n", len, size);
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }
}

char local_char;
err = MPI_Scatter(input_word, 1, MPI_CHAR,
                  &local_char, 1, MPI_CHAR,
                  0, MPI_COMM_WORLD);
check_mpi_error(err, "MPI_Scatter failed");

int local_len = rank + 1;
char *local_str = (char *)malloc(local_len * sizeof(char));

if (local_str == NULL) {
    fprintf(stderr, "Malloc failed on rank %d\n", rank);
    MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
}

for (int i = 0; i < local_len; i++) {
    local_str[i] = local_char;
}

if (rank == 0) {
    recvcunts = (int *)malloc(size * sizeof(int));

```

```

    displs = (int *)malloc(size * sizeof(int));

    if (!recvcounts || !displs) {
        fprintf(stderr, "Malloc failed for Gatherv arrays\n");
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }

    int current_disp = 0;
    for (int i = 0; i < size; i++) {
        recvcounts[i] = i + 1;
        displs[i] = current_disp;
        current_disp += recvcounts[i];
    }

    final_result = (char *)malloc((current_disp + 1) *
sizeof(char));
    if (!final_result) {
        fprintf(stderr, "Malloc failed for final_result\n");
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }
    final_result[current_disp] = '\0';
}

err = MPI_Gatherv(local_str, local_len, MPI_CHAR,
                  final_result, recvcounts, displs, MPI_CHAR,
                  0, MPI_COMM_WORLD);
check_mpi_error(err, "MPI_Gatherv failed");

if (rank == 0) {
    printf("Output : %s\n", final_result);

    free(input_word);
    free(final_result);
    free(recvcounts);
    free(displs);
}

// Cleanup Local Memory
free(local_str);

err = MPI_Finalize();
check_mpi_error(err, "MPI_Finalize failed");

return 0;
}

```

Output:

```
● STUDENT@MIT-ICT-LAB5-13:~/Desktop/230905380/Week4$ mpicc q4.c -o q4
● STUDENT@MIT-ICT-LAB5-13:~/Desktop/230905380/Week4$ mpirun -np 4 ./q4
  Enter a word of length 4: PCAP
  Output : PCCAAAPPPP
○ STUDENT@MIT-ICT-LAB5-13:~/Desktop/230905380/Week4$ █
```